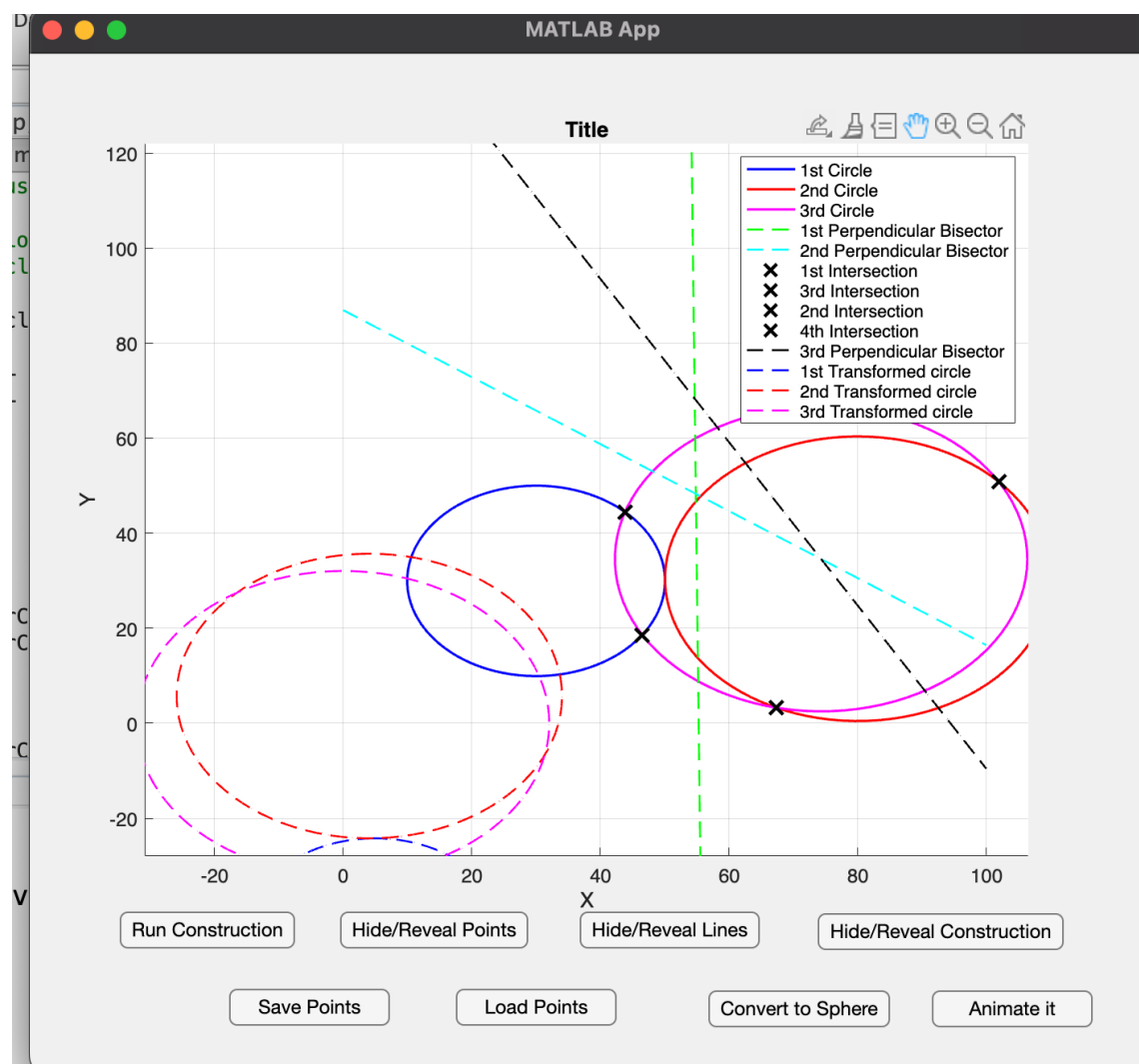


23025758 Student

The assignment was based on creating an interactive GUI app which allows user to place points on the axes and draw multiple circles based on some geometric rules. After creating the first two circles which second circle lies on the circumference of the first, we were prompted to draw a third circle and output perpendicular lines of these circles. The app with my code will then calculate and plot the intersections of the circles (Task 5). And lastly it will transform the circles by first translating the third circle at the origin and rotating the rest 90 degrees.

I added 4 buttons up to this point (Task 6) The run construction button, Hide/Reveal points, lines and constructions. Until Task 7 which prompted me to add 2 more buttons for saving and loading points. The last thing I was given was to add some exciting features to the app. The features I could do are creating 3D spheres out of the circles. Below is a picture of my program running



When the app starts running, user can place points to create circles. The user needs to input at least 6 points for the run construction button to work or else will be given an error indicting to do so. The app calculates lines called perpendicular bisectors, which

help find the centres of the second and third circles. These lines also help with the transformation of the circles. The program finds the intersection points of the circles to show their relationships clearly. Additionally, the app has buttons to save and load points. Users can easily save their points to a file and load them again later, making it easier to test and use the same points without re-entering them.

I improved the basic algorithms in the stubs to make them work and more accurate. For example, in `radiusTangentialCircle.m` I ensured it correctly calculates the radius of the second circle based on the first circle's centre and a point on its edge. In `intersections2circles.m`, I added checks to handle different cases for circle intersections, so it can find zero, one, or two points where circles meet. I also updated `closestPointOnCircle.m` to find the nearest circle to a point, helping the app figure out the closest point on the right circle.

For Task 8, I added a feature that turns the third circle into a 3D sphere, which users can activate by clicking a dedicated button. The sphere has realistic lighting, shading, and proper size adjustments. I also included animation for the geometric elements, so now the plotting process is animated, making the tool more interactive and fun to use. This adds a dynamic way for users to explore geometric shapes in both 2D and 3D.

